



Redes Neurais Artificiais

Profa. Jasmine Araújo
jasmine@ufpa.br



Introdução

- ◆ Existem uma série de problemas que os seres vivos e os seres humanos, em particular, parecem resolver de uma maneira inata, utilizando seu cérebro:
 - Processamento de imagens
 - Reconhecimento de fala
 - Recuperação de informações de maneira associativa
 - Aprendizado de novos fatos e idéias
 - Seleção de informações

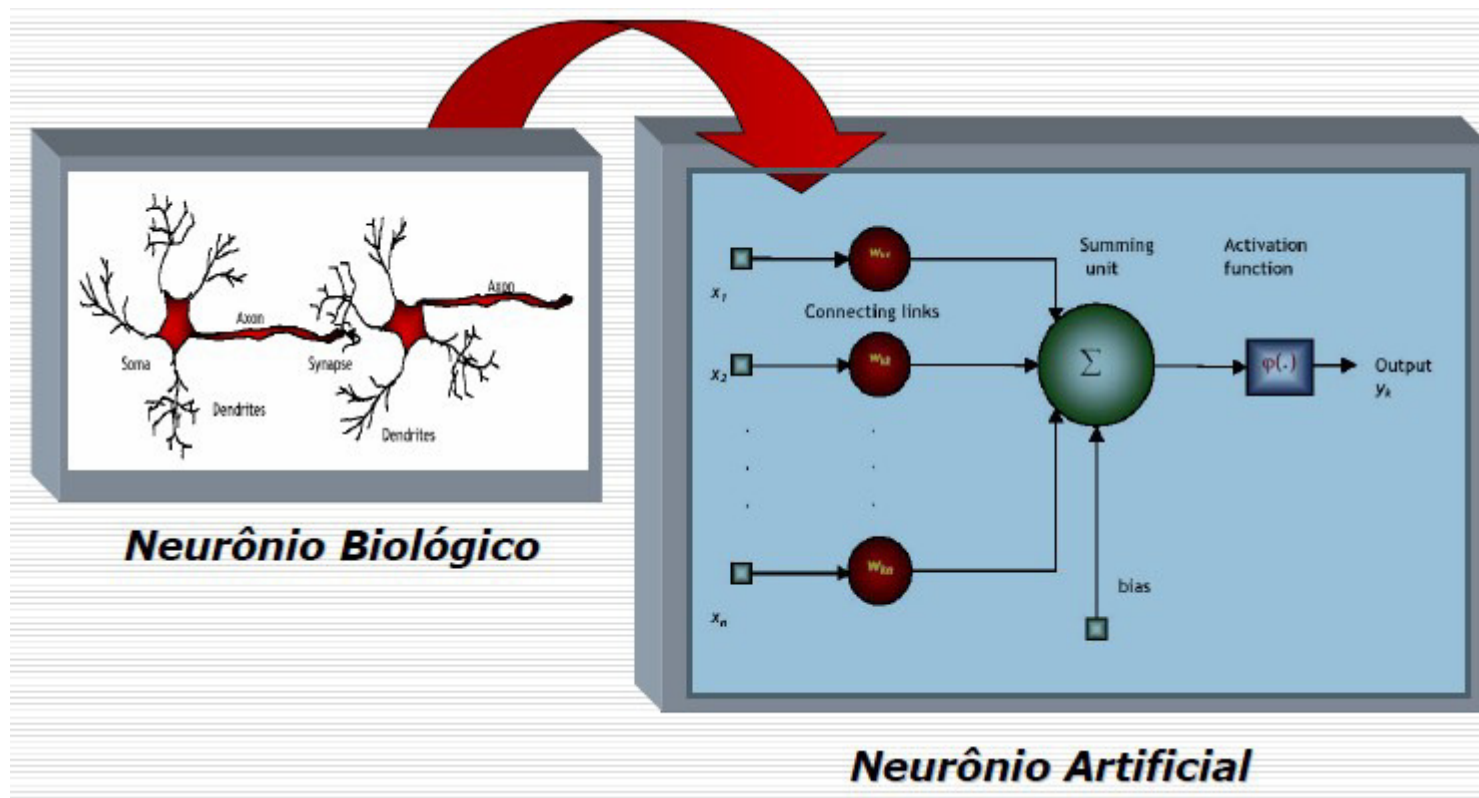


O que são Redes Neurais Artificiais?

- ◆ São modelos computacionais, inspirados nos neurônios biológicos e na estrutura do cérebro, com capacidade de:
 - adquirir, armazenar e utilizar conhecimento experimental.
 - provê mecanismos de generalização
 - conseguem agrupar ou organizar dados
 - eficientes onde métodos tradicionais têm se mostrado inadequados
- ◆ Compostas por várias unidades de processamento (“neurônios”).
- ◆ Interligadas por um grande número de conexões (“sinapses”).



Neurônio Artificial



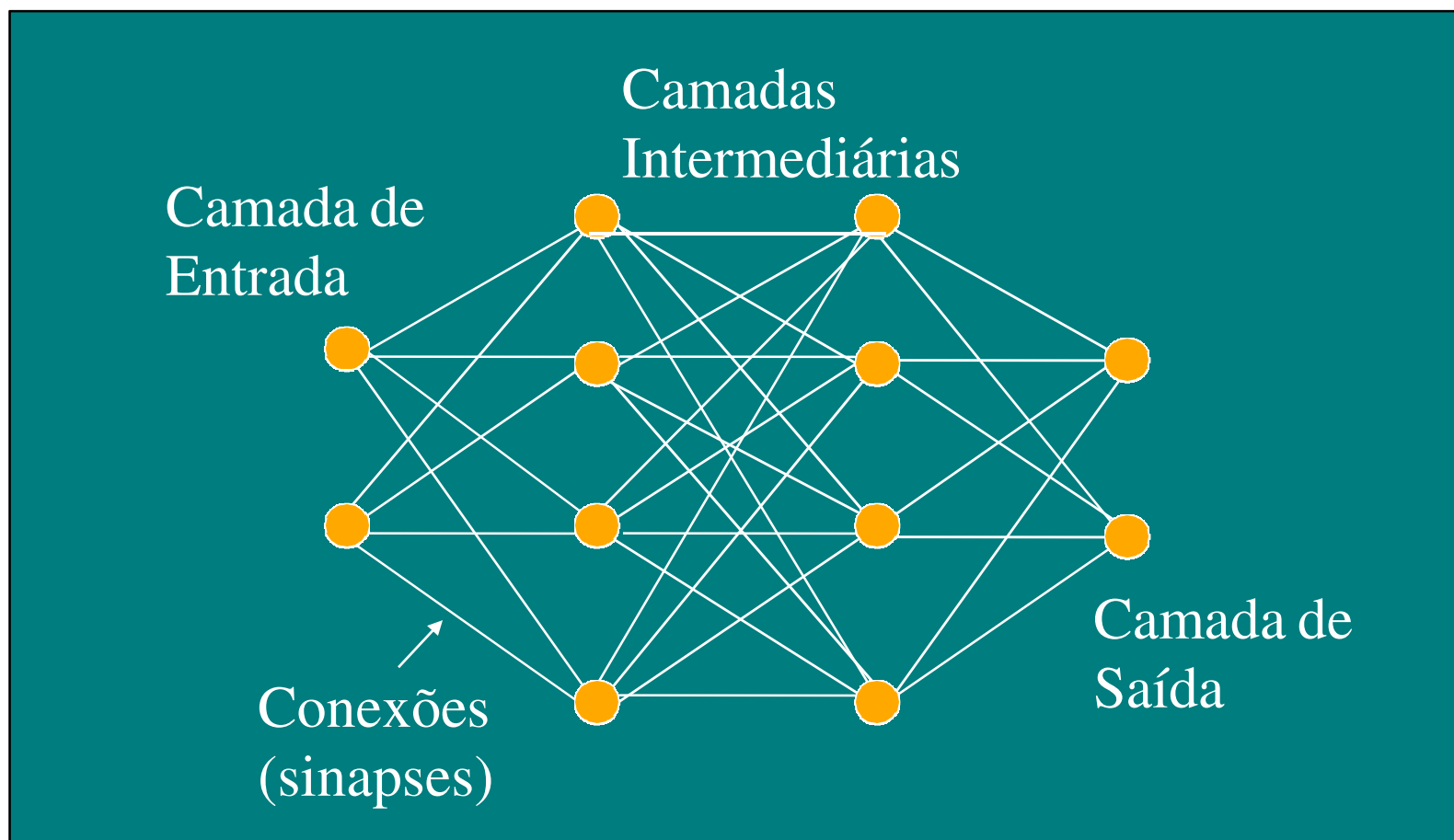


Rede Neural Artificial

- ♦♦ Uma rede neural artificial (RNA) é um sistema composto por vários neurônios, com estes neurônios sendo interconectados e normalmente agrupados em camadas
- ♦♦ *Alguns neurônios recebem excitações do exterior*
 - *neurônios de entrada*
 - *formam a camada de entrada*
- ♦♦ *Alguns neurônios transmitem sinais para o exterior*
 - *neurônios de saída*
 - *formam a camada de saída*
- ♦♦ *Alguns neurônios não recebem nada do exterior nem transmitem nada para o exterior*
 - *neurônios internos (escondidos)*
 - *formam a camada intermediária (escondida)*



Rede Neural Artificial





Rede Neural Artificial

Importantes Propriedades e Capacidades:

- ♦ Generalização – a rede neural pode produzir saídas adequadas para entradas que não estavam presentes durante o treinamento (aprendizagem).
- ♦ Não-linearidade – um neurônio artificial pode ser linear ou não linear.
- ♦ Mapeamento entrada-saída – aprendizado realizado através de amostras/ exemplos de treinamento (sinal de entrada e sinal de saída); a RNA aprende dos exemplos ao construir o mapeamento entrada-saída.
- ♦ Adaptabilidade – as redes neurais apresentam capacidade inata de adaptar seus pesos sinápticos a modificações no meio ambiente.
- ♦ Resposta a evidências – uma rede neural pode ser treinada não somente para fornecer a classificação de determinado padrão, mas também para fornecer o grau de confiança na decisão tomada



Histórico RNA

- ♦♦ 1943 - McCulloch & Pitts desenvolvem modelo matemático de RNA
- ♦♦ 1969 - Minsky & Papert analisam a rede Perceptron e mostram que elas não resolveriam alguns problemas bem simples
- ♦♦ 1982 - Hopfield mostra que Redes Neurais podem ser tratadas como sistemas dinâmicos
- ♦♦ 1986 - Hinton, Rumelhart e Williams propoem algoritmo de aprendizagem para redes multicamadas



Estrutura de uma RNA

♦ Estrutura geral das RNAs:

- Unidades de processamento n_i (nós ou neurônios)
 - Somador
 - Função de Ativação
 - Saída
- Conexões W_{ij}
- Topologia
 - Uma camada
 - Multicamada



Unidades de Processamento

- ◆◆ Cada sinapse (conexão) é caracterizada por um peso. Especificamente, um sinal x_j na entrada da sinapse j conectada ao neurônio k é multiplicada pelo peso sináptico w_{kj}
- ◆◆ O somador adiciona os sinais de entrada, ponderados pelos respectivos pesos das sinapses do neurônio



Unidades de Processamento

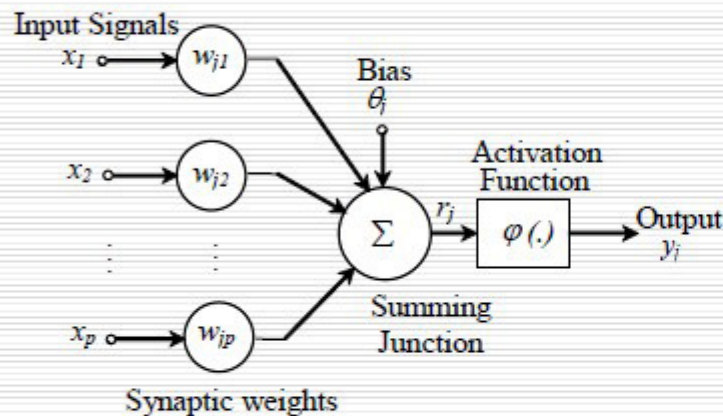
- ❖ Uma função de ativação para restringir a amplitude da saída do neurônio. Tipicamente, o intervalo normalizado da amplitude da saída de um neurônio é definido entre $[0, 1]$ ou $[-1, 1]$
- ❖ O modelo de neurônio inclui também um *bias* aplicado externamente, representado por θ_k . O bias tem o efeito de aumentar ou diminuir a entrada da função de ativação



Neurônio Artificial

Modelo Matemático Geral do Neurônio

Unidade de Processamento:



Formado basicamente por:

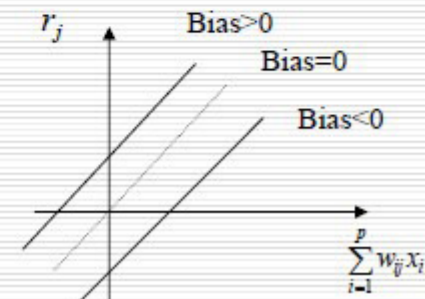
- Sinapses
- Somador
- Função Ativação

Matematicamente:

$$r_j = \sum_{i=1}^p w_{ij} x_i + \theta_j \quad (1)$$

$$y_j = \varphi\left(\sum_{i=1}^p w_{ij} x_i + \theta_j\right) \quad (2)$$

Bias – aumentar ou diminuir a entrada da função ativação, dependendo se ele é positivo ou negativo



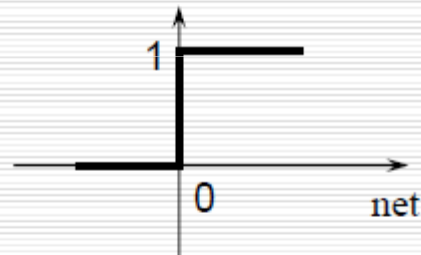


Neurônio Artificial

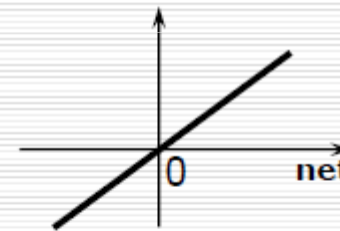
Modelo Matemático Geral do Neurônio

♦ Funções de Ativação

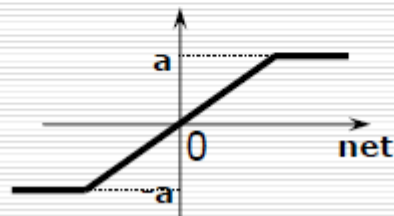
1. Limiar: $y = \begin{cases} 1 & \text{se } net > 0 \\ 0 & \text{se } net \leq 0 \end{cases}$



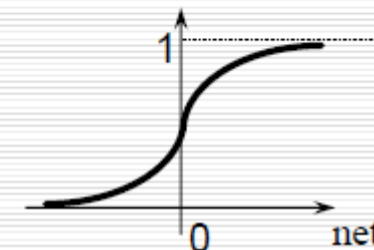
3. Linear: $y = a \times net$



Limiar por partes: $y = \begin{cases} a & \text{se } net \geq a \\ net & \text{se } -a < net < a \\ -a & \text{se } net \leq -a \end{cases}$



4. Sigmóide: $y = \frac{1}{1 + e^{(-a \times net + b)}}$





Conexões

◆ Tipo de conexões

- Excitatória: $w_{ik} > 0$
- Inibitória: $w_{ik} < 0$
- Inexistente: $w_{ik} = 0$

◆ Número de conexões de um nó

- Fan-in: número de conexões de entrada
- Fan-out: número de conexões de saída



Rede Neural Artificial

Aprendizagem

- ❖ É um processo pelo qual os parâmetros livres de uma rede neural são adaptados
- ❖ O tipo de aprendizagem é determinado pela maneira pela qual a modificação dos parâmetros ocorre
- ❖ Redes Neurais podem ser usadas tanto em problemas que possibilitem aprendizado supervisionado, quanto não-supervisionado



Rede Perceptron de Camada Única



Rede Neural Artificial Perceptron

- ◆ Rede mais simples para classificação de padrões linearmente separáveis
- ◆ Utiliza modelo de McCulloch-Pitts como neurônio
- ◆ Consiste de um único neurônio com pesos sinápticos ajustáveis e *bias*



RNA Treinamento

♦♦Supervisionado

♦♦Por correção de erro

$$\blacksquare w_{ij} = w_{ij} + \Delta w_{ij} \quad \text{para } (d \neq y)$$

$$\blacksquare \Delta w_{ij} = \eta x_i (d_j - y_j) \quad \text{para } (d \neq y)$$

$$\blacksquare \Delta w_{ij} = 0 \quad \text{para } (d = y)$$

■Onde η é a taxa de aprendizagem

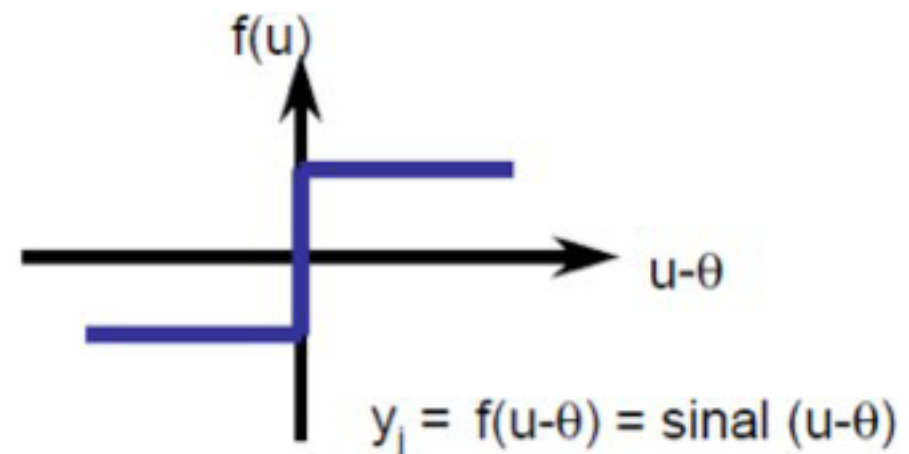


RNA Treinamento

- ♦♦Resposta / saída da rede
- ♦♦Aplica função sobre soma ponderada do total de entradas do nó

$$u = \sum_{i=1}^N x_i w_{ij}$$

$$y_j = \begin{cases} +1 & \text{if } u \geq \theta \\ -1 & \text{if } u < \theta \end{cases}$$





Algoritmo de Treinamento

Iniciar todas as conexões

Repita

Para cada par de treinamento (X, d)

Calcular a saída y

Se $(d \neq y)$

Então

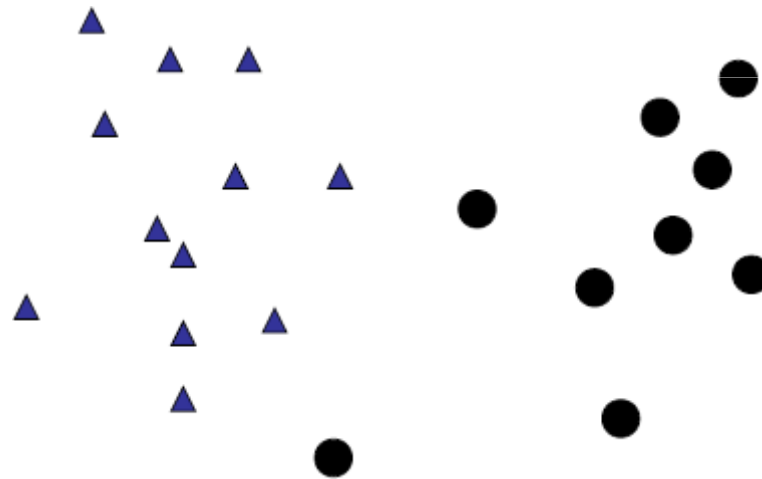
Atualizar pesos

Até erro aceitável (critério de parada)



Rede Neural Artificial Perceptron

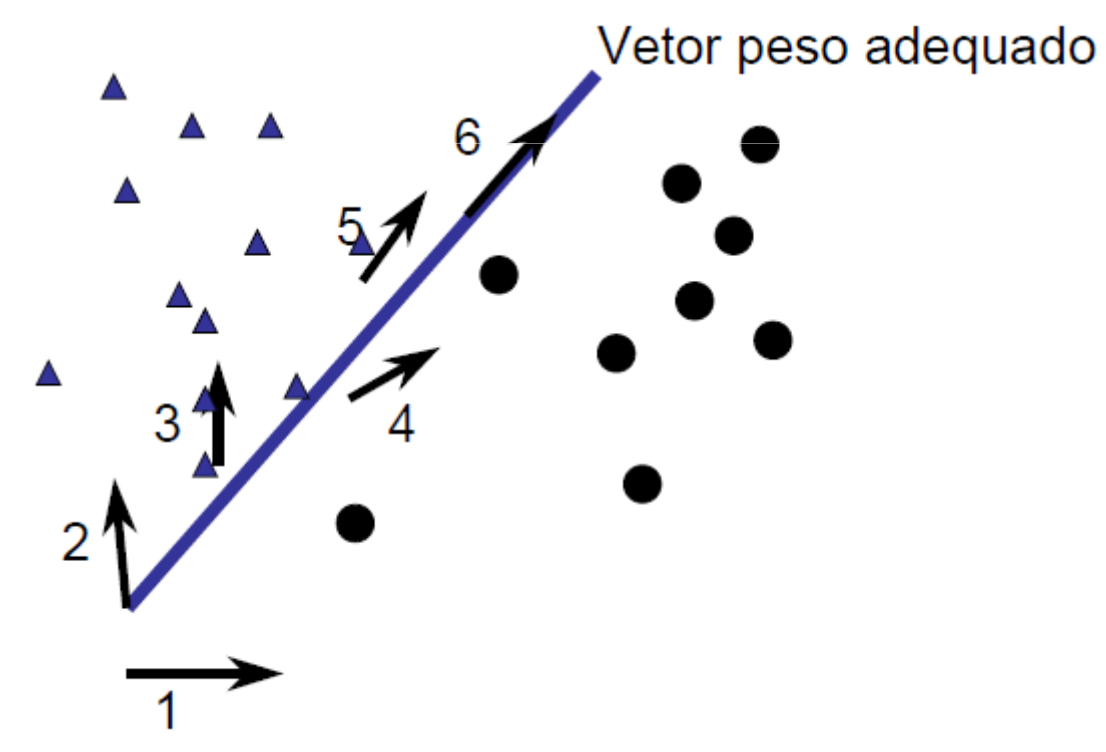
Algoritmo de Treinamento (espaço de busca da solução)





Rede Neural Artificial Perceptron

Algoritmo de Treinamento (espaço de busca da solução)





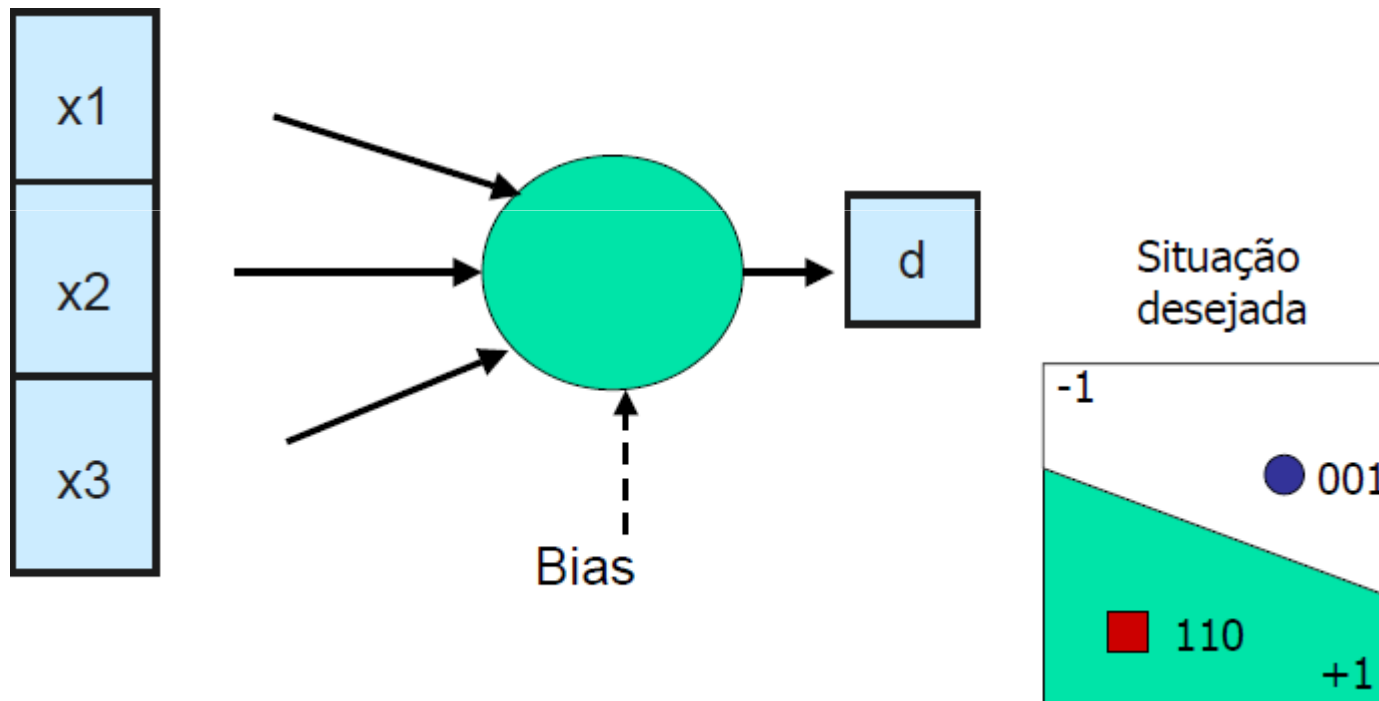
RNA Exemplo

❖ Dada uma rede Perceptrons formada por três entradas, utilizando pesos iniciais $w_0 = 0,4$, $w_1 = -0,6$, $w_2 = 0,6$, $w_3 = 0,5$ (peso bias), *bias* = -1 e uma taxa de aprendizado = 0,4, responda:

- Ensinar a rede a gerar a saída -1 para o padrão 001 e a saída 1 para os padrões 110
- A que classe pertencem os padrões 111, 000, 100, 011?

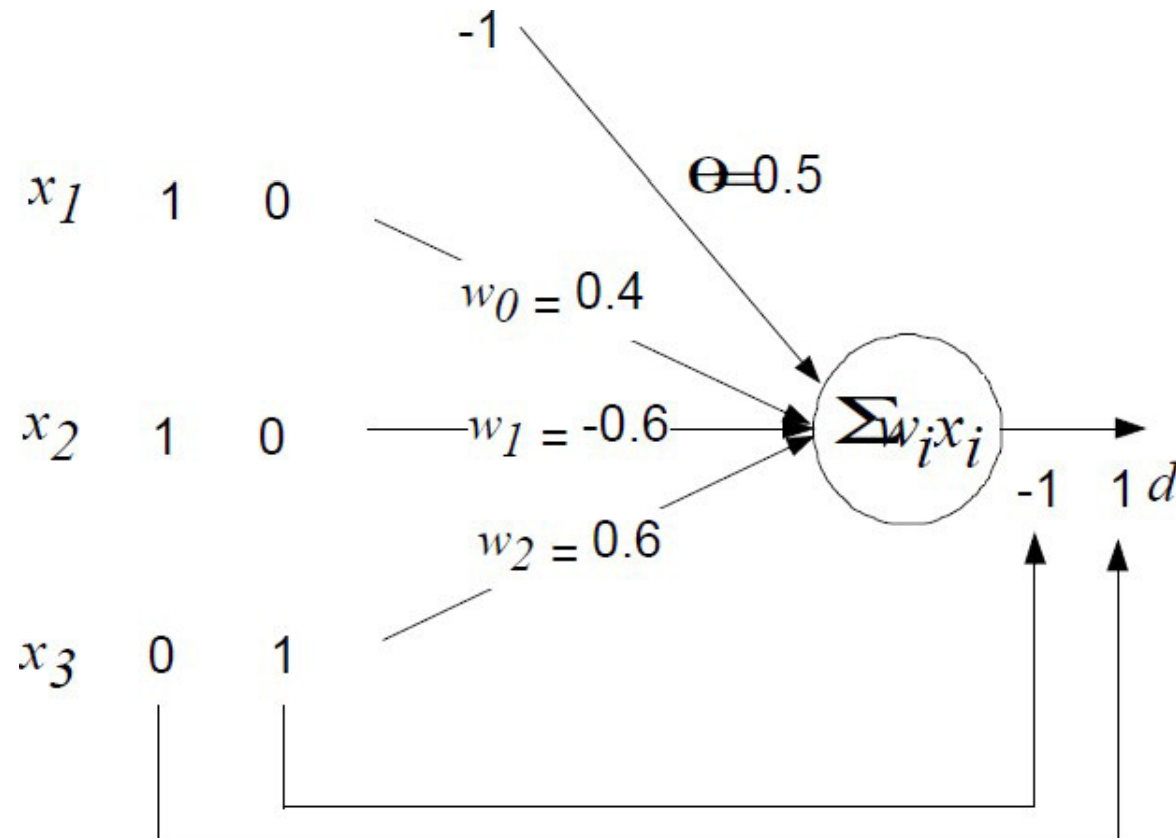


RNA Exemplo





RNA Exemplo





RNA Exemplo

a) Treinar a rede

a.1) Para o padrão 001 (d = -1)

Passo 1: definir a saída da rede

$$u = 0(0.4) + 0(-0.6) + 1(0.6) - 1(0.5) = 0.1$$

$$y = u = +1 \text{ (uma vez } 0.1 \geq 0 \text{)}$$

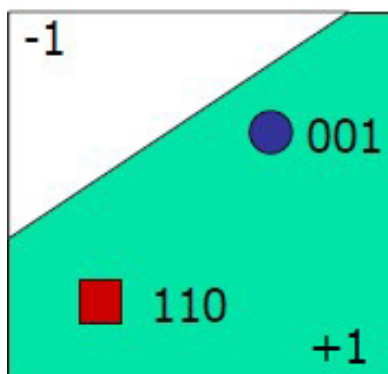
Passo 2: atualizar pesos ($d \neq y$)

$$w_0 = 0.4 + 0.4(0)(-1 - (+1)) = 0.4$$

$$w_1 = -0.6 + 0.4(0)(-1 - (+1)) = -0.6$$

$$w_2 = 0.6 + 0.4(1)(-1 - (+1)) = -0.2$$

$$w_3 = 0.5 + 0.4(-1)(-1 - (+1)) = 1.3$$





RNA Exemplo

a) Treinar a rede

a.2) Para o padrão 110 (d = +1)

Passo 1: definir a saída da rede

$$u = 1(0.4) + 1(-0.6) + 0(-0.2) - 1(1.3) = -1.5$$

$$y = u = -1 \text{ (uma vez } -1.5 < 0 \text{)}$$

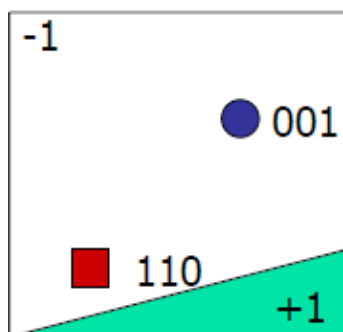
Passo 2: atualizar pesos ($d \neq y$)

$$w_0 = 0.4 + 0.4(1)(1 - (-1)) = 1.2$$

$$w_1 = -0.6 + 0.4(1)(1 - (-1)) = 0.2$$

$$w_2 = -0.2 + 0.4(0)(1 - (-1)) = -0.2$$

$$w_3 = 1.3 + 0.4(-1)(1 - (-1)) = 0.5$$





RNA Exemplo

a) Treinar a rede

a.3) Para o padrão 001 ($d = -1$)

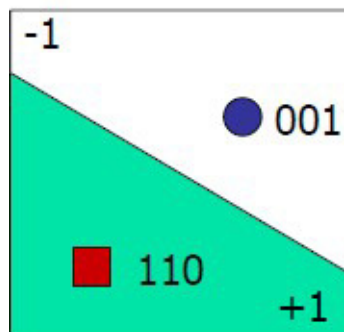
Passo 1: definir a saída da rede

$$u = 0(1.2) + 0(0.2) + 1(-0.2) - 1(0.5) = -0.7$$

$$y = u = -1 \text{ (uma vez } -0.7 < 0 \text{)}$$

Passo 2: atualizar pesos ($d = y$)

Como $d = y$, os pesos não precisam ser modificados





RNA Exemplo

a) Treinar a rede

a.4) Para o padrão 110 (d = +1)

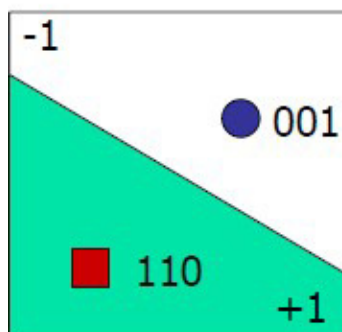
Passo 1: definir a saída da rede

$$u = 1(1.2) + 1(0.2) + 0(-0.2) - 1(0.5) = +0.7$$

$$y = u = +1 \text{ (uma vez } 0.7 > 0 \text{)}$$

Passo 2: atualizar pesos (d = y)

Como d = y, os pesos não precisam ser modificados





RNA Exemplo

b) Testar a rede

b.1) Para o padrão 111

$$u = 1(1.2) + 1(0.2) + 1(-0.2) - 1(0.5) = 0.7$$

$$y = u = 1 \text{ (porque } 0.7 \geq 0 \text{)} \Rightarrow \text{classe 1}$$

b.2) Para o padrão 000

$$u = 0(1.2) + 0(0.2) + 0(-0.2) - 1(0.5) = -0.5$$

$$y = u = -1 \text{ (porque } -0.5 < 0 \text{)} \Rightarrow \text{classe 0}$$



RNA Exemplo

b) Testar a rede

b.3) Para o padrão 100

$$u = 1(1.2) + 0(0.2) + 0(-0.2) + 1(-0.5) = 0.7$$

$$y = u = 1 \text{ (porque } 0.7 \geq 0) \Rightarrow \text{classe 1}$$

b.4) Para o padrão 011

$$u = 0(1.2) + 1(0.2) + 1(-0.2) - 1(0.5) = -0.5$$

$$y = u = -1 \text{ (porque } -0.5 < 0) \Rightarrow \text{classe 0}$$



Rede Neural Artificial Perceptron

Problemas com o Perceptron

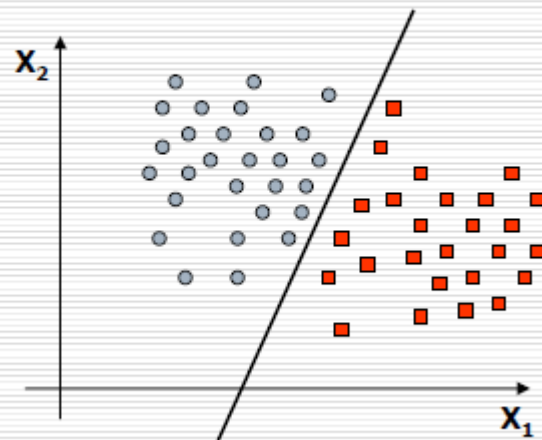
- Redes Perceptron de uma camada resolvem apenas problemas linearmente separáveis
- Grande número de aplicações importantes não são linearmente separáveis



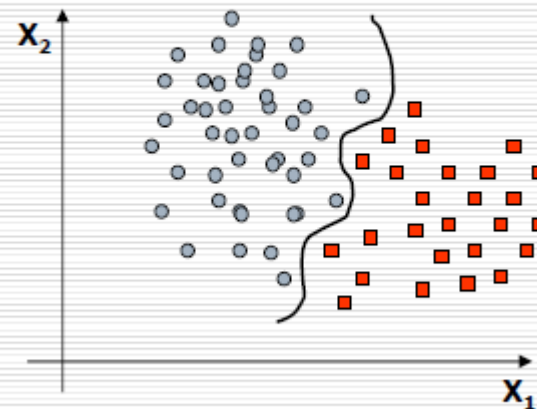
Rede Neural Artificial Perceptron

Problemas com o Perceptron (tipos de problemas)

Problemas linearmente separáveis são aqueles que podem ser resolvidos utilizando uma reta ou hiperplano como fronteira de decisão.



Linearmente separável



Não-linearmente separável



♦Dúvidas???